

Krampus RPG — AI Agent Operating Guide

Repository: CountKrampus/krampusrpg

This is the canonical repository-local instruction file for AI coding agents working on CountKrampus/krampusrpg.

Read this file before changing code. The goal is to keep every AI agent aligned with the same architecture, roadmap, security rules, data invariants, and development workflow.

1. Project

Krampus RPG is a browser-based, persistent Pokémon-inspired RPG.

Repository: CountKrampus/krampusrpg

Primary branch: main

Backend: Python / Flask

Database: SQLite

Python: 3.10+

Entry point: run.py

Database initializer: init_db.py

The long-term target is a persistent online RPG with accounts, owned Pokémon, exploration, encounters, catching, training, battles, evolution, economy, quests, regions, variants, trading, marketplace, social systems, and eventually PvP.

The project should feel like one coherent game, not a collection of disconnected demo pages.

2. Current Architecture

Important current areas include:

- run.py — Flask application entry point.
- init_db.py — database initialization and seeding.
- Server/app.py — Flask application factory and routes.
- Server/database.py — schema, migrations, seeding, Pokémon storage repair, and invariants.
- Server/auth.py — authentication.
- Server/config.py — configuration and paths.
- Server/services.py — Pokémon/game services.
- Server/party_storage.py — Party storage.
- Server/pc_storage.py — PC storage.
- Server/pc_routes.py — PC routes.
- Server/news.py — news system.
- Server/admin/routes.py — administration.
- Web/templates/ — templates.
- Web/static/ — CSS, JavaScript, and sprites.
- Data/ — JSON-backed game data where applicable.

Current documented dependencies are intentionally small: Flask and Werkzeug.

Do not add dependencies casually.

3. Development Philosophy

Build in dependency order.

1. Application foundation
2. Database/persistence
3. Accounts/authentication
4. Player state
5. Species/Pokémon data
6. Pokémon ownership
7. Party/PC
8. Areas/encounters
9. Catching
10. Training/battles
11. Items/healing
12. Progression/evolution
13. Economy
14. Trading/marketplace
15. Quests/story
16. Regions/world
17. PvP
18. Social/community
19. Events/operations/polish

Do not jump randomly between features when a lower-level dependency is unfinished.

Persistent game state must be server-side and database-backed. Temporary scaffolding is acceptable during implementation, but it must not become the permanent source of truth.

4. Critical Pokémon Storage Invariant

The authoritative architecture is:

```
pokemon
-> party
-> pc_storage
```

The old `pokemon.is_active` approach is NOT the authoritative Party system.

Every owned Pokémon must be in exactly one authoritative storage location:

- Party
- OR PC

Never allow:

- Party and PC simultaneously
- neither Party nor PC
- another player's storage
- invalid Party slots

The Party limit is 6.

The repository contains repair and verification logic for these rules. Preserve it.

If a Pokémon is orphaned, the intended repair behavior is to put it in the owner's PC.

If a Pokémon exists in both Party and PC, Party takes priority and the PC duplicate must be removed.

If storage ownership differs from `pokemon.owner_id`, correct the storage record rather than trusting it.

Do not introduce new code using `pokemon.is_active` as Party truth.

5. Current Pokémon Stat Philosophy

The current application intentionally does NOT use the old IV/EV/Nature/permanent-status design.

Do not casually reintroduce:

- IVs
- EVs
- Nature
- Permanent Status

Future progression mechanics should be intentionally designed as Krampus RPG mechanics instead of being accidentally copied from another system.

6. Pokémon Concepts

Keep these concepts separate.

Species:

The base species/template.

Owned Pokémon:

A specific creature owned by a player. It can have a unique ID, species, level, XP, HP, gender, ability, moves, variant, shiny state, owner, storage location, and progression state.

Variant:

A custom visual/theme/color variant of a species.

The project's variant concepts include:

- normal
- ruby
- sapphire
- emerald
- gold

- silver
- undead
- future custom variants

These names represent custom Krampus RPG color/theme variants, not games or generations of origin.

Shiny is separate from variant.

Conceptually:

```
species = gastly
```

```
variant = undead
```

```
shiny = false
```

is different from:

```
species = gastly
```

```
variant = normal
```

```
shiny = true
```

and the system may eventually support both being true when appropriate assets exist.

Do not collapse variant and shiny into one field.

7. Sprite Rules

Sprites are assets and should be referenced, not overwritten.

Filename conventions include:

pikachu.png

pikachu-shiny.png

pikachu-ruby.png

pikachu-ruby-shiny.png

pikachu-sapphire.png

pikachu-emerald.png

pikachu-gold.png

pikachu-silver.png

When generating variants:

- preserve the original sprite;
- create a separate asset;
- use deterministic filenames;
- preserve transparency;
- preserve the target dimensions;
- avoid unnecessary quality loss;
- never overwrite the source image.

Existing project work supports automatically recoloring many Pokémon into custom variants.

Some custom sprite tasks use dimensions such as 68x56 or 86x56. Do not assume either dimension globally; inspect the target component first.

8. Design Direction

The project should be as similar to the Eclipse-style online Pokémon RPG experience as practical while remaining an original game.

The intended experience includes:

- persistent accounts;
- persistent Pokémon collections;
- regions/routes/areas;
- exploration/searching;
- encounter tables;
- rarity;
- catching;
- training;
- healing;
- Party and PC management;
- evolution;
- shops;
- currency;
- trading;
- marketplace;
- quests;
- badges/progression;
- social features;
- eventually PvP.

This does NOT authorize copying proprietary code, assets, text, maps, or other copyrighted material.

Create original species, lore, mechanics, artwork, UI, maps, story, and balancing.

9. Roadmap

Phase 0 — Foundation

- Flask application
- configuration
- SQLite
- schema/migrations
- initialization
- templates/static assets

- health check
- basic error handling
- structured game data

This foundation exists and is being expanded.

Phase 1 — Accounts and Player

- registration
- login
- logout
- password hashing
- profiles
- progress
- sessions
- roles/permissions
- secure authorization

There is a webmaster-level permission concept.

IMPORTANT: the news admin/editor is webmaster-only. Ordinary players and generic staff must not receive news editing permissions unless the permission design is intentionally changed.

Phase 2 — Pokémon Ownership

- species
- owned Pokémon
- unique IDs
- owner relationship
- level
- XP
- HP
- gender
- ability
- moves
- shiny
- custom variant
- evolution state
- storage state

Starter Pokémon must be real owned Pokémon.

Do not allow a second starter merely because the first starter is in PC storage. Ownership determines whether a starter has already been selected.

Phase 3 — Party and PC

- six-slot Party
- PC storage
- Party -> PC
- PC -> Party
- reorder
- PC pages/slots
- ownership validation
- repair
- invariant checks

This is foundational and must remain reliable.

Phase 4 — World and Exploration

- regions
- areas/routes
- requirements
- unlocks
- searching
- encounter tables
- rarity
- wild Pokémon generation

Keep encounter data structured rather than hard-coded into templates.

Phase 5 — Catching

- capture items
- capture calculations
- success/failure
- ownership creation
- storage assignment
- capture logs
- rarity
- shiny/variant generation

A catch must never create an orphaned Pokémon.

Phase 6 — Training and Battles

- training battles
- wild battles
- battle state

- turn order
- moves
- damage
- accuracy/effects
- XP
- level-up
- fainting
- healing
- battle logs

Core calculations belong server-side and must be testable.

Phase 7 — Progression

- leveling
- evolution
- evolution items
- friendship/familiarity equivalent if designed
- time/location requirements where appropriate
- forms/variants
- progression rewards

The existing evolution-rule infrastructure should be extended rather than replaced with a competing system.

Phase 8 — Items and Economy

- currency
- inventory
- items
- Poké Balls
- healing
- battle items
- evolution items
- TMs/skills/items as designed
- shops
- buying
- selling
- transaction logs

All money-changing operations must be server-side and transactional.

Never trust client-supplied prices, balances, quantities, or ownership.

Phase 9 — Quests and Story

- quest definitions
- player quest state
- objectives
- requirements
- completion
- rewards
- story progression
- NPC interactions
- regional progression

Existing JSON-backed quest seeding should remain compatible unless intentionally migrated.

Phase 10 — Trading and Marketplace

- direct trades
- confirmation
- ownership transfer
- validation
- logs
- cancellation/timeout
- marketplace listings
- purchases
- anti-duplication protection

Use database transactions for asset transfers.

Phase 11 — Regions and World Expansion

- towns
- routes
- regions
- special areas
- NPCs
- encounters
- unlocks
- badges
- story arcs
- regional economies

World content should be data-driven so additional regions do not require rewriting the engine.

Phase 12 — Social

Potential systems include:

- friends
- profiles
- trading
- marketplace
- rankings
- records
- awards
- events
- forums/community
- referrals

Build persistent systems rather than UI-only placeholders.

Phase 13 — PvP

Eventually support:

- player battles
- matchmaking
- battle records
- competitive rules
- rankings
- seasonal/event systems

Reuse the core battle engine where practical.

Phase 14 — Live Game / Operations

Eventually add:

- daily rewards
- events
- achievements
- statistics
- moderation
- audit logs
- anti-cheat/bot detection
- backups
- monitoring
- performance work
- mobile polish

- accessibility

10. Agent Rules

Inspect before editing

Before changing a subsystem:

1. Read the relevant source.
2. Search all references.
3. Inspect database tables.
4. Inspect routes/templates/services.
5. Identify migrations/repair code.
6. Identify the authoritative source of truth.
7. Check permissions.

Do not implement based only on a filename.

Never fabricate repository facts

Do not claim a file, route, database table, feature, API, migration, or test result exists unless it has been verified.

Never say a test passed unless it was actually run.

Prefer complete file replacements

The project workflow strongly prefers full file replacements rather than tiny snippets.

When giving code to the project owner:

- provide the complete replacement file when practical;
- give the exact path;
- do not make the user guess where a snippet belongs;
- do not silently omit surrounding code.

Keep Python valid

A previous setup-script failure happened because Markdown/folder-tree text was accidentally placed into a Python file, producing Pylance syntax errors.

Therefore:

- Python files contain Python only.
- Markdown belongs in .md files.
- folder trees belong in documentation.
- never append explanatory prose to Python source.
- validate generated scripts before declaring them usable.

Database first

If state must survive refresh, logout, restart, or another device, it belongs in persistent server-side state.

Keep game logic out of templates

Templates render state. Core rules belong in services/database/game modules.

Validate all client input

Never trust:

- player IDs
- Pokémon IDs
- species IDs
- item IDs
- prices
- quantities
- balances
- battle outcomes
- inventory
- Party slots
- storage locations
- permissions

Authentication is not authorization

Being logged in does not authorize:

- admin functions
- news editing
- editing other players
- moving other players' Pokémon
- awarding currency
- modifying game data

Check permissions explicitly.

Never expose secrets

Never commit:

- GitHub PATs
- API keys
- passwords
- production secrets
- database credentials
- session secrets

Use environment/configuration mechanisms.

Safe migrations

Database initialization can be run repeatedly.

Schema changes should be idempotent where practical, explicit, tested, and safe for existing player data.

Never delete player data merely to simplify implementation.

11. Admin Rules

Administrative functionality must use the permission system.

The news editor is webmaster-only.

Admin routes must:

- require authentication;
- require the correct permission;
- validate all inputs;
- log sensitive operations when appropriate;
- never expose arbitrary database manipulation to normal users.

12. Security

Assume every browser request is hostile/manipulable.

Use:

- password hashing;
- server-side authorization;
- parameterized SQL;
- ownership checks;
- quantity validation;
- transactional economy operations;
- duplicate-action protection;
- trade validation;
- server-side battle calculations.

Never build SQL using untrusted strings.

Never rely on hidden HTML fields for authorization.

Never trust JavaScript for currency, battle results, or ownership.

13. Data-Driven Design

Prefer structured data/database records for:

- species
- moves
- abilities

- encounters
- items
- quests
- evolution rules
- areas
- regions
- variants

Avoid giant route functions containing hundreds of hard-coded game definitions.

The project should be able to grow to a large collection without rewriting the engine.

14. Custom Pokémon / Species

Original creatures and undead/ghost-inspired variants are part of the project's creative direction.

When adopting a species, define as appropriate:

- name
- types
- stats
- abilities
- moves
- evolution
- rarity
- variants
- sprites
- encounter data

Do not leave a conversational concept disconnected from actual project data when it has been approved for implementation.

15. UI Direction

The UI should feel like a persistent RPG website.

Prioritize:

- clear navigation;
- player identity;
- Pokémon visibility;
- Party visibility;
- meaningful actions;
- readable battle state;
- responsive layouts;
- mobile usability;

- consistent visual language.

Existing coming-soon pages are scaffolding, not proof of completed features.

16. Current Route Caveat

The application currently has routes for many planned areas, including concepts such as:

- Pokédex
- trades
- friends
- staff
- rules
- forums
- records
- awards
- referrals
- battle arena
- story adventure
- world exploration
- mines
- Pokémon Center
- minigames
- research
- exchanges
- training
- plaza
- marketplace
- locations
- events
- inventory
- settings
- daily rewards
- rankings
- Pokémon market
- trading

Many currently route to a generic coming-soon page.

The existence of a route is NOT proof that a feature is implemented.

Verify the route, service, database, templates, and tests before treating a feature as complete.

17. Testing

Before declaring a change complete:

- run a Python syntax/compile check on modified files;
- initialize the database;
- run the application or relevant test suite;
- test the affected route/service;
- check logs/errors;
- test invalid and unauthorized inputs where relevant.

For a modified Python file, a useful minimum is:

```
python -m py_compile path/to/file.py
```

Application startup is:

```
python init_db.py
```

```
python run.py
```

Only report checks that were actually performed.

18. Change Workflow

Understand

Identify existing implementation, dependencies, schema, UI, permissions, assets, and migration behavior.

Design

Decide the authoritative state, database representation, service API, route/API behavior, UI behavior, validation, and failure cases.

Implement

Change the smallest coherent set of files. Avoid unrelated refactors.

Verify

Test normal paths, invalid input, unauthorized access, persistence, repeated actions, and edge cases.

Document

If the architecture or rules change, update this file.

19. Git Rules

Prefer focused commits.

Examples:

- feat: add encounter generation
- feat: add party pc transfer
- fix: repair orphaned pokemon storage
- fix: enforce webmaster news permissions

- refactor: centralize pokemon ownership validation
- docs: update agent guide

Do not rewrite history or force-push unless explicitly instructed.

Do not delete player data to make development easier.

20. When Existing Code Looks Wrong

Do not immediately rewrite it.

Determine whether it is:

1. active logic;
2. compatibility logic;
3. migration logic;
4. repair logic;
5. deprecated but still referenced;
6. an actual bug.

The database contains deliberate migration/repair behavior. Preserve recovery paths unless they are demonstrably obsolete.

21. Conflict Resolution

Priority:

1. Explicit current user request
2. Existing security/data invariants
3. This AGENTS.md
4. Existing architecture/conventions
5. General coding preferences

If a requested feature conflicts with a critical data invariant, implement it in a way that preserves the invariant.

If a request risks corrupting or deleting player data, stop before the destructive operation and explain the specific risk.

22. Handoff Protocol

When finishing a task, report:

Changed

Files changed and what they do.

Database

whether schema/data changed.

Verification

Exactly what was run.

Remaining

Anything intentionally not implemented.

Next

The next dependency-driven task.

This allows another AI agent to continue without reconstructing the entire conversation.

23. Current High-Level State

The repository currently has a Flask foundation with:

- account registration/login/logout;
- player profiles;
- SQLite persistence;
- role/permission infrastructure;
- Pokémon ownership;
- species/owned-Pokémon infrastructure;
- Party/PC storage;
- starter selection;
- Pokémon variants;
- news infrastructure;
- admin routing;
- quest seeding;
- evolution-rule infrastructure;
- many planned feature routes;
- sprite assets and custom variant support.

Large parts of the eventual RPG remain under construction.

Do not interpret foundation code as proof that the full game is complete.

24. Product Vision

The finished game should let a player:

1. create an account;
2. choose a starter;
3. explore the world;
4. encounter creatures;
5. catch them;
6. build a persistent collection;
7. manage a six-Pokémon Party;
8. store additional Pokémon in the PC;

9. train and battle;
10. level and evolve;
11. obtain shiny/custom variants;
12. complete quests;
13. earn/spend currency;
14. buy/sell items;
15. trade with players;
16. use a marketplace;
17. progress through regions;
18. earn badges/achievements;
19. participate in events;
20. eventually participate in PvP.

The architecture must optimize for persistent state, extensibility, data integrity, and incremental development.

25. Final Agent Checklist

Before editing:

- ☐ Read AGENTS.md.
- ☐ Inspect relevant files.
- ☐ Search references.
- ☐ Inspect affected database tables.
- ☐ Identify the source of truth.
- ☐ Check permissions.
- ☐ Check for an existing implementation.
- ☐ Avoid duplicate systems.

Before finishing:

- ☐ Preserve Party/PC invariants.
- ☐ Preserve ownership integrity.
- ☐ Validate input.
- ☐ Keep persistent state server-side.
- ☐ Preserve original sprites.
- ☐ Do not expose secrets.
- ☐ Run relevant verification.
- ☐ Report exactly what was verified.
- ☐ Update AGENTS.md if architecture/rules changed.

Most Important Rule

Do not treat Krampus RPG as a collection of pages.

Treat it as a persistent game system.

Every feature should answer:

- What is the authoritative state?
- Where is it persisted?
- Who is allowed to change it?
- What happens if the request is repeated or manipulated?
- How does it interact with the rest of the game?

If an agent answers those questions before implementing features, Krampus RPG can grow from its current foundation into the intended full RPG without accumulating contradictory systems.